

DATABASE_AND_SECURITY_REVIEW.md — Revisão de Banco e Segurança

Revisão detalhada do banco de dados (Supabase/PostgreSQL), das políticas de RLS, do Storage, das permissões e dos riscos de segurança do **Born to Run**. Baseada em `supabase/schema.sql` (fonte da verdade), `lib/actions/*` e `middleware.ts`. Inclui inconsistências, melhorias, possíveis alterações de schema e um plano de migração. **Nenhum segredo é exposto neste documento.**

Sumário do veredito

- **Modelagem:** sólida e coesa; um bom ativo do projeto.
 - **RLS:** habilitada em todas as tabelas, majoritariamente correta, com **3 falhas relevantes** (DELETE em `profiles` ausente, `WITH CHECK` faltando em UPDATE admin, privacidade de treinos inexistente vs. prometida).
 - **Storage:** buckets bem configurados.
 - **Maior risco transversal:** cliente Supabase **sem tipos Database**, o que esconde erros de coluna até o runtime.
 - **Segredos:** nenhuma chave real versionada; `.env*` no `.gitignore`; `.env.example` provido.
-

1. Tabelas Existentes

Todas em `public`, com RLS habilitada.

1.1 profiles

Coluna	Tipo	Notas
id	UUID PK	uuid_generate_v4()
user_id	UUID	UNIQUE , FK → auth.users(id) ON DELETE CASCADE
full_name	TEXT	default ''
avatar_url	TEXT	nullable
bio	TEXT	nullable
cidade	TEXT	nullable
objetivo	TEXT	nullable
role	TEXT	CHECK IN ('member','admin') — não existe treinador
created_at / updated_at	TIMESTAMPTZ	trigger de updated_at

1.2 posts

Coluna	Tipo	Notas
id	UUID PK	
user_id	UUID	FK → auth.users(id) CASCADE
caption	TEXT	nullable (não content)
photo_url	TEXT	nullable (não image_url)
distance_km	NUMERIC(6,2)	métrica de corrida
duration_minutes	INTEGER	
pace	TEXT	
created_at / updated_at	TIMESTAMPTZ	

1.3 comments

Coluna	Tipo	Notas
id	UUID PK	
post_id	UUID	FK → posts(id) CASCADE
user_id	UUID	FK → auth.users(id) CASCADE
content	TEXT	CHECK length 1-500
created_at	TIMESTAMPTZ	

1.4 likes

Coluna	Tipo	Notas
id	UUID PK	
post_id	UUID	FK → posts(id) CASCADE
user_id	UUID	FK → auth.users(id) CASCADE
—	—	UNIQUE(post_id, user_id) (impede like duplicado)
created_at	TIMESTAMPTZ	

1.5 workouts

Coluna	Tipo	Notas
id	UUID PK	
title	TEXT	not null
description	TEXT	not null
level	TEXT	CHECK IN ('iniciante','intermediario','avancado')
objective	TEXT	not null
scheduled_date	DATE	nullable
created_by	UUID	FK → auth.users(id) CASCADE
created_at / updated_at	TIMESTAMPTZ	

Não existe assigned_to . Não há noção de “treino privado por aluno” no schema.

1.6 announcements

Coluna	Tipo	Notas
id	UUID PK	
title	TEXT	not null
content	TEXT	not null
created_by	UUID	FK → auth.users(id) CASCADE
created_at / updated_at	TIMESTAMPTZ	

2. Relacionamentos

```
auth.users (Supabase Auth)
├── 1:1 (user_id UNIQUE)
│   └── profiles
├── 1:N (posts.user_id)
│   ├── posts ── 1:N ── comments (post_id)
│   │               └── 1:N ── likes (post_id, UNIQUE por user)
├── 1:N (workouts.created_by)
│   └── workouts
├── 1:N (announcements.created_by)
│   └── announcements
```

- **Observação importante:** os vínculos de `posts`, `workouts`, `comments`, `likes` e `announcements` são para `auth.users`, não para `profiles`. Portanto **não existem** as FKs `posts_created_by_fkey` nem `workouts_created_by_fkey` que o código quebrado referencia. Para exibir dados do autor, é preciso juntar `posts.user_id` / `workouts.created_by` com `profiles.user_id` explicitamente (não há relacionamento nomeado direto com `profiles`).

3. Migrations

- **Estado atual:** o banco é definido por um **único arquivo** `supabase/schema.sql` (não há pasta de migrações versionadas).
- **Riscos:** sem histórico de migrações, é difícil evoluir o schema com segurança, reproduzir ambientes e reverter mudanças.
- **Recomendação:** adotar `supabase/migrations/` com migrações incrementais e numeradas (ex.: `0001_init.sql`, `0002_profiles_delete_policy.sql`, ...), mantendo o `schema.sql` como snapshot ou substituindo-o pelo fluxo de migrações do Supabase CLI.

4. Inconsistências Encontradas

#	Inconsistência	Origem	Impacto
1	Papel <code>treinador</code> não existe (CHECK só <code>member / admin</code>)	HANDOVER e <code>app/actions/workouts.ts</code> dependem dele	Código morto/ quebrado
2	Colunas <code>posts.content / posts.image_url</code> não existem (são <code>caption / photo_url</code>)	HANDOVER, <code>feed / dashboard</code> , <code>/fotos</code> , <code>app/actions/post.ts</code>	Feed e fotos quebrados
3	Coluna <code>workouts.assigned_time</code> não existe	HANDOVER, <code>/treinos</code> , <code>Create-WorkoutModal</code>	Promessa de privacidade falsa
4	FKs <code>posts_created_by_fk</code> / <code>workouts_created_by_fk</code> não existem	queries do código quebrado	Erros de runtime
5	Cliente Supabase sem tipos <code>Database</code>	setup dos clients	Erros de coluna só em runtime
6	<code>profiles</code> sem policy de DELETE	<code>schema.sql</code>	<code>deleteMember</code> falha silenciosa
7	<code>profiles_update_admin</code> sem <code>WITH CHECK</code>	<code>schema.sql</code>	Admin pode gravar fora do escopo
8	<code>workouts_select</code> USING (<code>true</code>)	<code>schema.sql</code>	Todos veem todos os treinos (contradiz UI)
9	<code>resetPassword</code> com <code>redirectTo</code> para domínio do Supabase	<code>lib/actions/auth.ts</code>	Reset de senha quebrado

5. Políticas RLS (análise detalhada)

5.1 profiles

- `profiles_select` — SELECT para `authenticated ( true )`: qualquer autenticado vê todos os perfis. **Aceitável** para uma comunidade de equipe; considerar limitar campos sensíveis se necessário.
- `profiles_update_own` — UPDATE próprio (`user_id = auth.uid()`) com `WITH CHECK`. 
- `profiles_update_admin` — UPDATE para admin (`is_admin()`), **sem** `WITH CHECK`.  Adicionar `WITH CHECK (public.is_admin())`.
- **DELETE — ausente.**  Adicionar `profiles_delete_admin ( FOR DELETE ... USING (public.is_admin()) )` para viabilizar remoção de membros. Avaliar também deletar o usuário em `auth.users` (via função server-side com `service_role`, fora do cliente) para remoção completa.

5.2 posts

- SELECT `true`; INSERT `user_id = auth.uid()`; UPDATE próprio; DELETE próprio **ou** admin.  Bem modelada.

5.3 comments

- SELECT `true`; INSERT próprio; DELETE próprio ou admin.  (Sem UPDATE — comentários imutáveis; ok.)

5.4 likes

- SELECT `true`; INSERT próprio; DELETE próprio; UNIQUE(post_id,user_id). 

5.5 workouts

- SELECT `true` (**todos veem tudo**); INSERT/UPDATE/DELETE só admin.  A leitura pública contradiz a UI que promete “treinos designados a você”. Decidir modelo (ver §7).

5.6 announcements

- SELECT `true`; INSERT/UPDATE/DELETE só admin. 

5.7 Funções de apoio

- `is_admin()` — SECURITY DEFINER, STABLE, checa `role='admin'` para `auth.uid()`.  Boa prática (evita recursão de RLS).
- `handle_new_user()` — SECURITY DEFINER, cria profile no signup. 
- Triggers de `updated_at`. 

6. Storage e Permissões

6.1 Buckets

- `avatars` — público, limite 5MB, MIME jpeg/jpg/png/webp.
- `post-images` — público, limite 10MB, MIME jpeg/jpg/png/webp.

6.2 Policies de Storage

- `avatars`: leitura pública; insert/update/delete restritos à **pasta do próprio usuário** (`storage.foldername(name))[1] = auth.uid()::text`). 

- `post-images` : leitura pública; insert por qualquer autenticado; delete pelo dono da pasta `ou admin`. ⚠️ `INSERT` não restringe a pasta ao `auth.uid()` — qualquer autenticado pode subir em qualquer caminho do bucket. Considerar restringir o insert por pasta do usuário (consistência com `avatars`).

6.3 Observações

- Buckets públicos expõem as URLs de leitura por padrão — adequado para avatares e fotos de feed públicas na comunidade, mas ciente de que qualquer um com a URL acessa o arquivo.
- Definir limpeza de arquivos órfãos (quando um post/perfil é excluído, remover a imagem correspondente do Storage) — hoje não há.

7. Possíveis Alterações de Schema

Decisões a tomar (documentar a escolha antes de migrar):

7.1 Modelo de papéis

- **Opção A (recomendada):** manter `member / admin` . Robson = `admin` . Simples e já suportado.
- **Opção B:** adicionar `treinador` ao `CHECK` e diferenciar permissões de admin vs treinador. Só se houver requisito real de separação; exige revisar todas as policies e o código.

7.2 Privacidade de treinos

- **Opção A (simples):** treinos são da equipe (todos os autenticados veem). Ajustar a **copy** para não prometer privacidade. Manter `workouts_select USING (true)` .
- **Opção B (privado por aluno):** adicionar `workouts.assigned_to UUID REFERENCES auth.users` , e RLS `workouts_select USING (assigned_to = auth.uid() OR public.is_admin())` . Exige UI de atribuição no admin e migração de dados.

7.3 Correções de RLS (independem das opções acima)

- Adicionar `profiles_delete_admin` .
- Adicionar `WITH CHECK` em `profiles_update_admin` .
- (Opcional) restringir `INSERT` em `post-images` por pasta do usuário.

7.4 Tipagem

- Gerar tipos `Database` do Supabase e aplicar aos clients (`createBrowserClient<Database>` / `createServerClient<Database>`), eliminando `any` e pegando erros de coluna em compile-time.

8. Plano de Migração (sugerido)

Aplicar em ambiente de **teste** do Supabase primeiro; validar com um usuário `member` e um `admin` ; só então promover a produção. Fazer backup antes.

1. `0001_init.sql` — snapshot do schema atual (baseline).
2. `0002_profiles_delete_policy.sql`

-

```
CREATE POLICY "profiles_delete_admin" ON public.profiles FOR DELETE TO authenticated USING (public.is_admin());
```


3. 0003_profiles_update_admin_withcheck.sql

- Recriar `profiles_update_admin` com `USING (public.is_admin()) WITH CHECK (public.is_admin());`

4. (Se Opção B de treinos) 0004_workouts_assigned_to.sql

- `ALTER TABLE public.workouts ADD COLUMN assigned_to UUID REFERENCES auth.users(id) ON DELETE CASCADE;`
- Recriar `workouts_select` com `USING (assigned_to = auth.uid() OR public.is_admin());`
- Índice em `assigned_to`.

5. (Se Opção B de papéis) 0005_role_treinador.sql

- Alterar o CHECK de `role` para incluir `'treinador'` e ajustar `is_admin()` /policies conforme a política de permissões.

6. (Opcional) 0006_post_images_insert_scope.sql

- Restringir INSERT em `post-images` à pasta do `auth.uid()`.

7. **Tipos:** gerar `Database types` e commitar em `types/` (ex.: `types/database.types.ts`), integrando aos clients.

Validação pós-migração (checklist):

- Admin remove membro → 1 linha afetada + feedback de sucesso.
 - Member tenta UPDATE de outro perfil → bloqueado.
 - (Opção B) Member vê só treinos designados; admin vê todos.
 - Upload de avatar/post continua funcionando dentro do escopo.
 - `tsc` acusa erro ao referenciar coluna inexistente (tipos ativos).
-

9. Riscos de Segurança e Mitigações

Risco	Severidade	Mitigação
Erros de coluna só em runtime (sem tipos)	🔴 Crítico	Gerar/aplicar tipos Database
Falha silenciosa em delete-Member	🟡 Alto	Policy DELETE + tratamento de erro com feedback
WITH CHECK ausente em UPDATE admin	🟡 Alto	Adicionar WITH CHECK
Treinos expostos a todos (vs. promessa)	🟡 Alto	Decidir modelo (A/B) e alinhar copy/RLS
INSERT amplo em post-images	🟡 Médio	Restringir por pasta do usuário
Middleware sem reforço de admin	🟡 Médio	Checar admin no middleware (defesa em profundidade)
Convenção middleware descontinuada	🟡 Médio	Migrar para proxy (Next 16)
service_role exposta no cliente	🔴 Crítico (se ocorrer)	Nunca usar service_role no cliente; só em server/edge seguro
Arquivos órfãos no Storage	🟢 Baixo	Rotina de limpeza ao excluir post/perfil
Vulnerabilidades transitivas (postcss)	🟡 Médio	Atualizar dependências com cautela

10. Segredos e Variáveis de Ambiente

- Variáveis usadas no código: `NEXT_PUBLIC_SUPABASE_URL` e `NEXT_PUBLIC_SUPABASE_ANON_KEY` — ambas **públicas por natureza** (prefixo `NEXT_PUBLIC_`), seguras para o cliente **porque a segurança real está na RLS**.
- `SUPABASE_SERVICE_ROLE_KEY` (se um dia necessária para operações administrativas server-side, como excluir usuário em `auth.users`): **jamais** com prefixo `NEXT_PUBLIC_`, **jamais** no cliente, **jamais** versionada. Usar apenas em código de servidor confiável.
- `.env*` está no `.gitignore`; o único arquivo versionado é `.env.example` (com placeholders). A auditoria **não** encontrou chaves reais no repositório.
- **Regra:** a segurança do Supabase depende da RLS, não do sigilo da `anon key`. Portanto, manter as policies corretas é a defesa principal.

11. Conclusão

O banco é o **ativo mais bem construído** do projeto: modelagem coesa, RLS real e Storage adequado. As correções necessárias são **cirúrgicas** (policy DELETE, WITH CHECK , decisão de privacidade de treinos) e devem vir acompanhadas da **tipagem Database** , que é a mudança de maior impacto para a qualidade — pois transforma dezenas de bugs de runtime em erros de compilação. Executar essas mudanças na **Fase 2** do FABLE_REBUILD_PLAN.md , com migrações versionadas e validação de RLS por papel.

Fim do DATABASE_AND_SECURITY_REVIEW.md.